9
Testing Concurrent Applications

In this chapter, we will cover the following topics:

- Monitoring a Lock interface
- Monitoring a Phaser class
- Monitoring an Executor framework
- Monitoring a fork/join pool
- Monitoring a stream
- Writing effective log messages
- Analyzing concurrent code with FindBugs
- Configuring Eclipse for debugging concurrency code
- Configuring NetBeans for debugging concurrency code
- Testing concurrency code with MultithreadedTC
- Monitoring with JConsole

Introduction

Testing an application is a critical task. Before you make an application ready for end users, you have to demonstrate its correctness. You use a test process to prove that correctness is achieved and errors are fixed. Testing is a common task in any software development and quality assurance process. You can find a lot of literature about testing processes and the different approaches you can apply to your developments. There are a lot of libraries as well, such as JUnit, and applications, such as Apache JMeter, that you can use to test your Java applications in an automated way. Testing is even more critical in concurrent application development.

The fact that concurrent applications have two or more threads that share data structures and interact with each other adds more difficulty to the testing phase. The biggest problem you will face when you test concurrent applications is that the execution of threads is non-deterministic. You can't guarantee the order of the execution of threads, so it's difficult to reproduce errors.

Monitoring a Lock interface

A `Lock` interface is one of the basic mechanisms provided by the Java concurrency API to synchronize a block of code. It allows you to define a **critical section**. A critical section is a block of code that accesses a shared resource and can't be executed by more than one thread at the same time. This mechanism is implemented by the `Lock` interface and the `ReentrantLock` class.

In this recipe, you will learn what information you can obtain about a `Lock` object and how to obtain that information.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `MyLock` that extends the `ReentrantLock` class:

```
public class MyLock extends ReentrantLock {
```

2. Implement `getOwnerName()`. This method returns the name of the thread that has control of a lock (if any), using the protected method of the `Lock` class called `getOwner()`:

```
public String getOwnerName() {
  if (this.getOwner()==null) {
    return "None";
  }
  return this.getOwner().getName();
}
```

3. Implement `getThreads()`. This method returns a list of threads queued in a lock, using the protected method of the `Lock` class called `getQueuedThreads()`:

```
public Collection<Thread> getThreads() {
  return this.getQueuedThreads();
}
```

4. Create a class named `Task` that implements the `Runnable` interface:

```
public class Task implements Runnable {
```

5. Declare a private `Lock` attribute named `lock`:

```
private final Lock lock;
```

6. Implement a constructor of the class to initialize its attribute:

```
public Task (Lock lock) {
  this.lock=lock;
}
```

7. Implement the `run()` method. Create a loop with five steps:

```
@Override
public void run() {
  for (int i=0; i<5; i++) {
```

8. Acquire the lock using the `lock()` method and print a message:

```
lock.lock();
System.out.printf("%s: Get the Lock.\n",
                  Thread.currentThread().getName());
```

9. Put the thread to sleep for 500 milliseconds. Free the lock using the `unlock()` method and print a message:

```
        try {
          TimeUnit.MILLISECONDS.sleep(500);
          System.out.printf("%s: Free the Lock.\n",
                            Thread.currentThread().getName());
        } catch (InterruptedException e) {
          e.printStackTrace();
        } finally {
          lock.unlock();
        }
      }
    }
  }
```

10. Create the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {
  public static void main(String[] args) throws Exception {
```

11. Create a `MyLock` object named `lock`:

```
MyLock lock=new MyLock();
```

12. Create an array of five `Thread` objects:

```
Thread threads[]=new Thread[5];
```

13. Create and start five threads to execute five `Task` objects:

```
for (int i=0; i<5; i++) {
  Task task=new Task(lock);
  threads[i]=new Thread(task);
  threads[i].start();
}
```

14. Create a loop with 15 steps:

```
for (int i=0; i<15; i++) {
```

15. Write the name of the owner of the lock in the console:

```
System.out.printf("Main: Logging the Lock\n");
System.out.printf("************************\n");
System.out.printf("Lock: Owner : %s\n",lock.getOwnerName());
```

16. Display the number and name of the threads queued for the lock:

```
System.out.printf("Lock: Queued Threads: %s\n",
                  lock.hasQueuedThreads());
if (lock.hasQueuedThreads()){
  System.out.printf("Lock: Queue Length: %d\n",
                    lock.getQueueLength());
  System.out.printf("Lock: Queued Threads: ");
  Collection<Thread> lockedThreads=lock.getThreads();
  for (Thread lockedThread : lockedThreads) {
    System.out.printf("%s ",lockedThread.getName());
  }
  System.out.printf("\n");
}
```

17. Display information about the fairness and status of the `Lock` object:

```
System.out.printf("Lock: Fairness: %s\n",lock.isFair());
System.out.printf("Lock: Locked: %s\n",lock.isLocked());
System.out.printf("************************\n");
```

18. Put the thread to sleep for 1 second and close the loop and the class:

```
      TimeUnit.SECONDS.sleep(1);
    }
  }
}
```

How it works...

In this recipe, you implemented the `MyLock` class that extends the `ReentrantLock` class to return information that wouldn't have been available otherwise-it's protected data of the `ReentrantLock` class. The methods implemented by the `MyLock` class are as follows:

- `getOwnerName()`: Only one thread can execute a critical section protected by a `Lock` object. The lock stores the thread that is executing the critical section. This thread is returned by the protected `getOwner()` method of the `ReentrantLock` class.

- `getThreads()`: When a thread is executing a critical section, other threads that try to enter it are put to sleep before they continue executing that critical section. The protected method `getQueuedThreads()` of the `ReentrantLock` class returns the list of threads that are waiting to execute the critical section.

We also used other methods that are implemented in the `ReentrantLock` class:

- `hasQueuedThreads()`: This method returns a `Boolean` value indicating whether there are threads waiting to acquire the calling `ReentrantLock`
- `getQueueLength()`: This method returns the number of threads that are waiting to acquire the calling `ReentrantLock`
- `isLocked()`: This method returns a `Boolean` value indicating whether the calling `ReentrantLock` is owned by a thread
- `isFair()`: This method returns a `Boolean` value indicating whether the calling `ReentrantLock` has fair mode activated

There's more...

There are other methods in the `ReentrantLock` class that can be used to obtain information about a `Lock` object:

- `getHoldCount()`: This returns the number of times the current thread has acquired the lock
- `isHeldByCurrentThread()`: This returns a `Boolean` value indicating whether the lock is owned by the current thread

See also

- The *Synchronizing a block of code with a lock* recipe in `Chapter 2`, *Basic Thread Synchronization*
- The *Implementing a custom Lock class* recipe in `Chapter 8`, *Customizing Concurrency Classes*

Monitoring a Phaser class

One of the most complex and powerful functionalities offered by the Java Concurrency API is the ability to execute concurrent-phased tasks using the `Phaser` class. This mechanism is useful when we have some concurrent tasks divided into steps. The `Phaser` class provides the mechanism to synchronize threads at the end of each step so no thread starts its second step until all the threads have finished the first one.

In this recipe, you will learn what information you can obtain about the status of a `Phaser` class and how to obtain that information.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task` that implements the `Runnable` interface:

```
public class Task implements Runnable {
```

2. Declare a private `int` attribute named `time`:

```
private final int time;
```

3. Declare a private `Phaser` attribute named `phaser`:

```
private final Phaser phaser;
```

4. Implement the constructor of the class to initialize its attributes:

```
public Task(int time, Phaser phaser) {
  this.time=time;
  this.phaser=phaser;
}
```

5. Implement the `run()` method. First, instruct the `phaser` attribute that the task starts its execution with the `arrive()` method:

```
@Override
public void run() {

  phaser.arrive();
```

6. Write a message in the console indicating the start of phase one. Put the thread to sleep for the number of seconds specified by the `time` attribute. Write a message in the console indicating the end of phase one. And, synchronize with the rest of the tasks using the `arriveAndAwaitAdvance()` method of the `phaser` attribute:

```
System.out.printf("%s: Entering phase 1.\n",
                  Thread.currentThread().getName());
try {
  TimeUnit.SECONDS.sleep(time);
} catch (InterruptedException e) {
  e.printStackTrace();
}
System.out.printf("%s: Finishing phase 1.\n",
                  Thread.currentThread().getName());
phaser.arriveAndAwaitAdvance();
```

7. Repeat this behavior in both second and third phases. At the end of the third phase, use the `arriveAndDeregister()` method instead of `arriveAndAwaitAdvance()`:

```
System.out.printf("%s: Entering phase 2.\n",
                  Thread.currentThread().getName());
try {
  TimeUnit.SECONDS.sleep(time);
} catch (InterruptedException e) {
  e.printStackTrace();
}
System.out.printf("%s: Finishing phase 2.\n",
                  Thread.currentThread().getName());
phaser.arriveAndAwaitAdvance();

System.out.printf("%s: Entering phase 3.\n",
                  Thread.currentThread().getName());
try {
  TimeUnit.SECONDS.sleep(time);
} catch (InterruptedException e) {
  e.printStackTrace();
}
```

```
System.out.printf("%s: Finishing phase 3.\n",
                  Thread.currentThread().getName());

phaser.arriveAndDeregister();
```

8. Implement the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {

  public static void main(String[] args) throws Exception {
```

9. Create a new `Phaser` object named `phaser` with three participants:

```
Phaser phaser=new Phaser(3);
```

10. Create and launch three threads to execute three task objects:

```
for (int i=0; i<3; i++) {
  Task task=new Task(i+1, phaser);
  Thread thread=new Thread(task);
  thread.start();
}
```

11. Create a loop with 10 steps to write information about the `phaser` object:

```
for (int i=0; i<10; i++) {
```

12. Write information about the registered parties, the phase of the `phaser`, the arrived parties, and the unarrived parties:

```
System.out.printf("********************\n");
System.out.printf("Main: Phaser Log\n");
System.out.printf("Main: Phaser: Phase: %d\n",
                  phaser.getPhase());
System.out.printf("Main: Phaser: Registered Parties: %d\n",
                  phaser.getRegisteredParties());
System.out.printf("Main: Phaser: Arrived Parties: %d\n",
                  phaser.getArrivedParties());
System.out.printf("Main: Phaser: Unarrived Parties: %d\n",
                  phaser.getUnarrivedParties());
System.out.printf("********************\n");
```

13. Put the thread to sleep for 1 second and close the loop and the class:

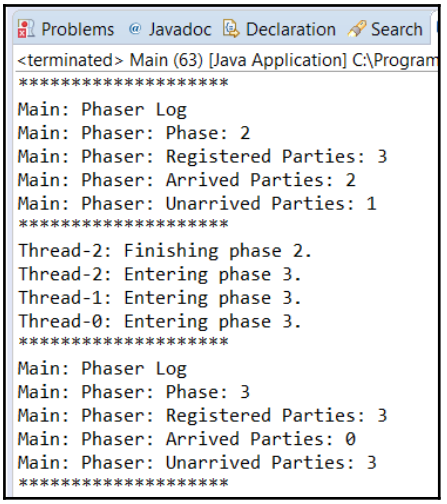
```
            TimeUnit.SECONDS.sleep(1);
        }
    }
}
```

How it works...

In this recipe, we implemented a phased task in the `Task` class. This phased task has three phases and uses a `Phaser` interface to synchronize with other `Task` objects. The main class launches three tasks, and when these tasks execute their respective phases, it prints information about the status of the `phaser` object to the console. We used the following methods to get the status of the `phaser` object:

- `getPhase()`: This method returns the actual phase of a `phaser` object
- `getRegisteredParties()`: This method returns the number of tasks that use a `phaser` object as a mechanism of synchronization
- `getArrivedParties()`: This method returns the number of tasks that have arrived at the end of the actual phase
- `getUnarrivedParties()`: This method returns the number of tasks that haven't yet arrived at the end of the actual phase

The following screenshot shows part of the output of the program:



See also

- The *Running concurrent-phased tasks* recipe in `Chapter 3`, *Thread Synchronization Utilities*

Monitoring an Executor framework

The `Executor` framework provides a mechanism that separates the implementation of tasks from thread creation and management to execute the tasks. If you use an executor, you only have to implement `Runnable` objects and send them to the executor. It is the responsibility of an executor to manage threads. When you send a task to an executor, it tries to use a pooled thread for executing the task in order to avoid the creation of new threads. This mechanism is offered by the `Executor` interface and its implementing classes as the `ThreadPoolExecutor` class.

In this recipe, you will learn what information you can obtain about the status of a `ThreadPoolExecutor` executor and how to obtain it.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task` that implements the `Runnable` interface:

```
public class Task implements Runnable {
```

2. Declare a private `long` attribute named `milliseconds`:

```
private final long milliseconds;
```

3. Implement the constructor of the class to initialize its attribute:

```
public Task (long milliseconds) {
  this.milliseconds=milliseconds;
}
```

4. Implement the `run()` method. Put the thread to sleep for the number of milliseconds specified by the `milliseconds` attribute:

```
@Override
public void run() {

  System.out.printf("%s: Begin\n",
                    Thread.currentThread().getName());
  try {
    TimeUnit.MILLISECONDS.sleep(milliseconds);
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
  System.out.printf("%s: End\n",
                    Thread.currentThread().getName());

}
```

5. Implement the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {

  public static void main(String[] args) throws Exception {
```

6. Create a new `Executor` object using the `newCachedThreadPool()` method of the `Executors` class:

```
ThreadPoolExecutor executor = (ThreadPoolExecutor)
                        Executors.newCachedThreadPool();
```

7. Create and submit 10 `Task` objects to the executor. Initialize the objects with a random number:

```
Random random=new Random();
for (int i=0; i<10; i++) {
  Task task=new Task(random.nextInt(10000));
  executor.submit(task);
}
```

8. Create a loop with five steps. In each step, write information about the executor by calling the `showLog()` method and putting the thread to sleep for a second:

```
for (int i=0; i<5; i++){
  showLog(executor);
  TimeUnit.SECONDS.sleep(1);
}
```

9. Shut down the executor using the `shutdown()` method:

```
executor.shutdown();
```

10. Create another loop with five steps. In each step, write information about the executor by calling the `showLog()` method and putting the thread to sleep for a second:

```
for (int i=0; i<5; i++){
  showLog(executor);
  TimeUnit.SECONDS.sleep(1);
}
```

11. Wait for the finalization of the executor using the `awaitTermination()` method:

```
executor.awaitTermination(1, TimeUnit.DAYS);
```

12. Display a message indicating the end of the program:

```
  System.out.printf("Main: End of the program.\n");
}
```

13. Implement the `showLog()` method that receives `Executor` as a parameter. Write information about the size of the pool, the number of tasks, and the status of the executor:

```
private static void showLog(ThreadPoolExecutor executor) {
  System.out.printf("**********************");
  System.out.printf("Main: Executor Log");
  System.out.printf("Main: Executor: Core Pool Size: %d\n",
                    executor.getCorePoolSize());
  System.out.printf("Main: Executor: Pool Size: %d\n",
                    executor.getPoolSize());
  System.out.printf("Main: Executor: Active Count: %d\n",
                    executor.getActiveCount());
  System.out.printf("Main: Executor: Task Count: %d\n",
                    executor.getTaskCount());
```

```
        System.out.printf("Main: Executor: Completed Task Count: %d\n",
                          executor.getCompletedTaskCount());
        System.out.printf("Main: Executor: Shutdown: %s\n",
                          executor.isShutdown());
        System.out.printf("Main: Executor: Terminating: %s\n",
                          executor.isTerminating());
        System.out.printf("Main: Executor: Terminated: %s\n",
                          executor.isTerminated());
        System.out.printf("**********************\n");
    }
```

How it works...

In this recipe, you implemented a task that blocks its execution thread for a random number of milliseconds. Then, you sent 10 tasks to an executor, and while you were waiting for their finalization, you wrote information about the status of the executor to the console. You used the following methods to get the status of the `Executor` object:

- `getCorePoolSize()`: This method returns an `int` number, which refers to the core number of threads. It's the minimum number of threads that will be in the internal thread pool when the executor is not executing any task.
- `getPoolSize()`: This method returns an `int` value, which refers to the actual size of the internal thread pool.
- `getActiveCount()`: This method returns an `int` number, which refers to the number of threads that are currently executing tasks.
- `getTaskCount()`: This method returns a `long` number, which refers to the number of tasks that have been scheduled for execution.
- `getCompletedTaskCount()`: This method returns a `long` number, which refers to the number of tasks that have been executed by this executor and have finished their execution.
- `isShutdown()`: This method returns a `Boolean` value when the `shutdown()` method of an executor is called to finish its execution.
- `isTerminating()`: This method returns a `Boolean` value when the executor performs the `shutdown()` operation but hasn't finished it yet.
- `isTerminated()`: This method returns a `Boolean` value when the executor finishes its execution.

See also

- The *Creating a thread executor and controlling its rejected tasks* recipe in `Chapter 4`, *Thread Executors*
- The *Customizing the ThreadPoolExecutor class* and *Implementing a priority-based Executor class* recipes in `Chapter 8`, *Customizing Concurrency Classes*

Monitoring a fork/join pool

The Executor framework provides a mechanism that allows you to separate task implementation from the creation and management of threads that execute the tasks. Java 9 includes an extension of the Executor framework for a specific kind of problem that will improve the performance of other solutions (using `Thread` objects directly or the Executor framework). It's the fork/join framework.

This framework is designed to solve problems that can be broken down into smaller tasks using the `fork()` and `join()` operations. The main class that implements this behavior is `ForkJoinPool`.

In this recipe, you will learn what information you can obtain about a `ForkJoinPool` class and how to obtain it.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task` that extends the `RecursiveAction` class:

```
public class Task extends RecursiveAction{
```

2. Declare a private `int` array attribute named `array` to store the array of elements you want to increment:

```
private final int array[];
```

3. Declare two private `int` attributes named `start` and `end` to store the start and end positions of the block of elements this task has to process:

```
private final int start;
private final int end;
```

4. Implement the constructor of the class to initialize its attributes:

```
public Task (int array[], int start, int end) {
  this.array=array;
  this.start=start;
  this.end=end;
}
```

5. Implement the `compute()` method with the main logic of the task. If the task has to process more than 100 elements, first divide the elements into two parts, create two tasks to execute these parts, start its execution with the `fork()` method, and finally, wait for its finalization with the `join()` method:

```
protected void compute() {
  if (end-start>100) {
    int mid=(start+end)/2;
    Task task1=new Task(array,start,mid);
    Task task2=new Task(array,mid,end);

    task1.fork();
    task2.fork();

    task1.join();
    task2.join();
```

6. If the task has to process 100 elements or less, increment the elements by putting the thread to sleep for 5 milliseconds after each operation:

```
  } else {
      for (int i=start; i<end; i++) {
        array[i]++;

        try {
          Thread.sleep(5);
        } catch (InterruptedException e) {
```

```
            e.printStackTrace();
          }
        }
      }
    }
  }
```

7. Implement the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {

  public static void main(String[] args) throws Exception {
```

8. Create a `ForkJoinPool` object named `pool`:

```
ForkJoinPool pool=new ForkJoinPool();
```

9. Create an array of integer numbers, named `array`, with 10,000 elements:

```
int array[]=new int[10000];
```

10. Create a new `Task` object to process the whole array:

```
Task task1=new Task(array,0,array.length);
```

11. Send the task for execution to the pool using the `execute()` method:

```
pool.execute(task1);
```

12. If the task doesn't finish its execution, call the `showLog()` method to write information about the status of the `ForkJoinPool` class and put the thread to sleep for a second:

```
while (!task1.isDone()) {
  showLog(pool);
  TimeUnit.SECONDS.sleep(1);
}
```

13. Shut down the pool using the `shutdown()` method:

```
pool.shutdown();
```

14. Wait for the finalization of the pool using the `awaitTermination()` method:

```
pool.awaitTermination(1, TimeUnit.DAYS);
```

15. Call the `showLog()` method to write information about the status of the `ForkJoinPool` class and write a message in the console indicating the end of the program:

```
showLog(pool);
System.out.printf("Main: End of the program.\n");
```

16. Implement the `showLog()` method. It receives a `ForkJoinPool` object as a parameter and writes information about its status and the threads and tasks that are being executed:

```
private static void showLog(ForkJoinPool pool) {
  System.out.printf("**********************\n");
  System.out.printf("Main: Fork/Join Pool log\n");
  System.out.printf("Main: Fork/Join Pool: Parallelism: %d\n",
                    pool.getParallelism());
  System.out.printf("Main: Fork/Join Pool: Pool Size: %d\n",
                    pool.getPoolSize());
  System.out.printf("Main: Fork/Join Pool: Active Thread Count:
                     %d\n", pool.getActiveThreadCount());
  System.out.printf("Main: Fork/Join Pool: Running Thread Count:
                     %d\n", pool.getRunningThreadCount());
  System.out.printf("Main: Fork/Join Pool: Queued Submission:
                     %d\n", pool.getQueuedSubmissionCount());
  System.out.printf("Main: Fork/Join Pool: Queued Tasks: %d\n",
                    pool.getQueuedTaskCount());
  System.out.printf("Main: Fork/Join Pool: Queued Submissions:
                     %s\n", pool.hasQueuedSubmissions());
  System.out.printf("Main: Fork/Join Pool: Steal Count: %d\n",
                    pool.getStealCount());
  System.out.printf("Main: Fork/Join Pool: Terminated : %s\n",
                    pool.isTerminated());
  System.out.printf("**********************\n");
}
```

How it works...

In this recipe, you implemented a task that increments the elements of an array, using a `ForkJoinPool` class, and a `Task` class that extends the `RecursiveAction` class. This is one of the tasks you can execute in a `ForkJoinPool` class. When the tasks were processing the array, you printed information about the status of the `ForkJoinPool` class to the console. You used the following methods to get the status of the `ForkJoinPool` class:

- `getPoolSize()`: This method returns an `int` value, which is the number of worker threads of the internal pool of a `ForkJoinPool` class
- `getParallelism()`: This method returns the desired level of parallelism established for a pool
- `getActiveThreadCount()`: This method returns the number of threads that are currently executing tasks
- `getRunningThreadCount()`: This method returns the number of working threads that are not blocked in any synchronization mechanism
- `getQueuedSubmissionCount()`: This method returns the number of tasks that have been submitted to a pool and haven't started their execution yet
- `getQueuedTaskCount()`: This method returns the number of tasks that have been submitted to a pool and have started their execution
- `hasQueuedSubmissions()`: This method returns a `Boolean` value indicating whether the pool has queued tasks that haven't started their execution yet
- `getStealCount()`: This method returns a `long` value specifying the number of times a worker thread has stolen a task from another thread
- `isTerminated()`: This method returns a `Boolean` value indicating whether the fork/join pool has finished its execution

See also

- The *Creating a fork/join pool* recipe in `Chapter 5`, *Fork/Join Framework*
- The *Implementing the ThreadFactory interface to generate custom threads for the fork/join framework* and *Customizing tasks running in the fork/join framework* recipes in `Chapter 8`, *Customizing Concurrency Classes*

Monitoring a stream

A stream in Java is a sequence of elements that could be processed (mapped, filtered, transformed, reduced, and collected) either parallelly or sequentially in a pipeline of declarative operations using `lambda` expressions. It was introduced in Java 8 to change the way one can process enormous sets of data in a functional way, with lambda expressions instead of the traditional imperative way.

The `Stream` interface doesn't provide a lot of methods as other concurrency classes to monitor its status. Only the `peek()` method allows you to write log information about the elements that are being processed. In this recipe, you will learn how to use this method to write information about a stream.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Main` with a `main()` method. Declare two private variables, namely an `AtomicInteger` variable called `counter` and a `Random` object called `random`:

```
public class Main {
  public static void main(String[] args) {

    AtomicLong counter = new AtomicLong(0);
    Random random=new Random();
```

2. Create a stream of 1,000 random `double` numbers. The stream created is a sequential stream. You have to make it parallel using the `parallel()` method, and use the `peek()` method to increment the value of the `counter` variable and write a message in the console. Post this, use the `count()` method to count the number of elements in the array and store that number in an integer variable. Write the value stored in the `counter` variable and the value returned by the `count()` method in the console:

```
long streamCounter = random.doubles(1000).parallel()
                     .peek( number -> {
  long actual=counter.incrementAndGet();
  System.out.printf("%d - %f\n", actual, number);
}).count();

System.out.printf("Counter: %d\n", counter.get());
System.out.printf("Stream Counter: %d\n", streamCounter);
```

3. Now, set the value of the `counter` variable to 0. Create another stream of 1,000 random `double` numbers. Then, convert it into a parallel stream using the `parallel()` method, and use the `peek()` method to increment the value of the `counter` variable and write a message in the console. Finally, use the `forEach()` method to write all the numbers and the value of the counter variable in the console:

```
    counter.set(0);
    random.doubles(1000).parallel().peek(number -> {
      long actual=counter.incrementAndGet();
      System.out.printf("Peek: %d - %f\n", actual,number);
    }).forEach( number -> {
      System.out.printf("For Each: %f\n", number);
    });

    System.out.printf("Counter: %d\n", counter.get());
  }
}
```

How it works...

In this example, we used the `peek()` method in two different situations to count the number of elements that pass by this step of the stream and write a message in the console.

As described in `Chapter 6`, *Parallel and Reactive Streams*, `Stream` has a source, zero or more intermediate operations, and a final operation. In the first case, our final operation is the `count()` method. This method doesn't need to process the elements to calculate the returned value, so the `peek()` method will never be executed. You won't see any of the messages of the peek method in the console, and the value of the counter variable will be 0.

The second case is different. The final operation is the `forEach()` method, and in this case, all the elements of the stream will be processed. In the console, you will see messages of both `peek()` and `forEach()` methods. The final value of the `counter` variable will be 1,000.

The `peek()` method is an intermediate operation of a stream. Like with all intermediate operations, they are executed lazily, and they only process the necessary elements. This is the reason why it's never executed in the first case.

See also

- The *Creating streams from different sources*, *Reducing the elements of a stream* and *Collecting the elements of a stream* recipes in `Chapter 6`, *Parallel and Reactive Streams*

Writing effective log messages

A **log** system is a mechanism that allows you to write information to one or more destinations. A **Logger** has the following components:

- **One or more handlers**: A handler will determine the destination and format of the log messages. You can write log messages in the console, a file, or a database.
- **A name**: Usually, the name of a Logger used in a class is based on the class name and its package name.
- **A level**: Log messages have different levels that indicate their importance. A Logger also has a level to decide what messages it is going to write. It only writes messages that are as important as, or more important, than its level.

You should use the log system because of the following two main reasons:

- Write as much information as you can when an exception is caught. This will help you localize the error and resolve it.
- Write information about the classes and methods that the program is executing.

In this recipe, you will learn how to use the classes provided by the `java.util.logging` package to add a log system to your concurrent application.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `MyFormatter` that extends the `java.util.logging.Formatter` class. Implement the abstract `format()` method. It receives a `LogRecord` object as a parameter and returns a `String` object with a log message:

```
public class MyFormatter extends Formatter {
  @Override
  public String format(LogRecord record) {

    StringBuilder sb=new StringBuilder();
    sb.append("["+record.getLevel()+"] - ");
    sb.append(new Date(record.getMillis())+" : ");
    sb.append(record.getSourceClassName()+ "."
              +record.getSourceMethodName()+" : ");
    sb.append(record.getMessage()+"\n");.
    return sb.toString();
  }
```

2. Create a class named `MyLoggerFactory`:

```
public class MyLoggerFactory {
```

3. Declare a private static `Handler` attribute named `handler`:

```
private static Handler handler;
```

4. Implement the public static method `getLogger()` to create the `Logger` object that you're going to use to write log messages. It receives a `String` parameter called `name`. We synchronize this method with the `synchronized` keyword:

```
public synchronized static Logger getLogger(String name){
```

5. Get `java.util.logging.Logger` associated with the name received as a parameter using the `getLogger()` method of the `Logger` class:

```
Logger logger=Logger.getLogger(name);
```

6. Establish the log level to write all the log messages using the `setLevel()` method:

```
logger.setLevel(Level.ALL);
```

7. If the handler attribute has the null value, create a new `FileHandler` object to write log messages in the `recipe8.log` file. Assign a `MyFormatter` object to this handler; assign it as a formatter using the `setFormatter()` object:

```
try {
  if (handler==null) {
    handler=new FileHandler("recipe6.log");
    Formatter format=new MyFormatter();
    handler.setFormatter(format);
  }
```

8. If the `Logger` object does not have a handler associated with it, assign the handler using the `addHandler()` method:

```
    if (logger.getHandlers().length==0) {
      logger.addHandler(handler);
    }
  } catch (SecurityException e | IOException e) {
  e.printStackTrace();
}
```

9. Return the `Logger` object created:

```
  return logger;
}
```

10. Create a class named `Task` that implements the `Runnable` interface. It will be the task used to test your `Logger` object:

```
public class Task implements Runnable {
```

11. Implement the `run()` method:

```
@Override
public void run() {
```

12. First, declare a `Logger` object named `logger`. Initialize it using the `getLogger()` method of the `MyLogger` class by passing the name of this class as a parameter:

```
Logger logger= MyLogger.getLogger(this.getClass().getName());
```

13. Write a log message indicating the beginning of the execution of the method, using the `entering()` method:

```
logger.entering(Thread.currentThread().getName(), "run()");
```

14. Sleep the thread for two seconds:

```
try {
  TimeUnit.SECONDS.sleep(2);
} catch (InterruptedException e) {
  e.printStackTrace();
}
```

15. Write a log message indicating the end of the execution of the method, using the `exiting()` method:

```
  logger.exiting(Thread.currentThread().getName(), "run()",
                 Thread.currentThread());
}
```

16. Implement the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

17. Declare a `Logger` object named `logger`. Initialize it using the `getLogger()` method of the `MyLogger` class by passing the `Core` string as a parameter:

```
Logger logger=MyLogger.getLogger(Main.class.getName());
```

18. Write a log message indicating the start of the execution of the main program, using the `entering()` method:

```
logger.entering(Main.class.getName(), "main()",args);
```

19. Create a `Thread` array to store five threads:

```
Thread threads[]=new Thread[5];
```

20. Create five `Task` objects and five threads to execute them. Write log messages to indicate that you're going to launch a new thread and that you have created the thread:

```
for (int i=0; i<threads.length; i++) {
  logger.log(Level.INFO,"Launching thread: "+i);
  Task task=new Task();
  threads[i]=new Thread(task);
  logger.log(Level.INFO,"Thread created: "+
            threads[i].getName());
  threads[i].start();
}
```

21. Write a log message to indicate that you have created the threads:

```
logger.log(Level.INFO,"Ten Threads created."+
           "Waiting for its finalization");
```

22. Wait for the finalization of the five threads using the `join()` method. After the finalization of each thread, write a log message indicating that the thread has finished:

```
for (int i=0; i<threads.length; i++) {
  try {
    threads[i].join();
    logger.log(Level.INFO,"Thread has finished its execution",
               threads[i]);
  } catch (InterruptedException e) {
    logger.log(Level.SEVERE, "Exception", e);
  }
}
```

23. Write a log message to indicate the end of the execution of the main program, using the `exiting()` method:

```
  logger.exiting(Main.class.getName(), "main()");
}
```

How it works...

In this recipe, you used the `Logger` class provided by the Java logging API to write log messages in a concurrent application. First of all, you implemented the `MyFormatter` class to assign a format to the log messages. This class extends the `Formatter` class that declares the abstract `format()` method. This method receives a `LogRecord` object with all of the information of the log message and returns a formatted log message. In your class, you used the following methods of the `LogRecord` class to obtain information about the log message:

- `getLevel()`: Returns the level of a message
- `getMillis()`: Returns the date when a message was sent to a `Logger` object
- `getSourceClassName()`: Returns the name of a class that had sent the message to the Logger
- `getSourceMessageName()`: Returns the name of the method that had sent the message to the Logger
- `getMessage()`: Returns the log message

The `MyLogger` class implements the static method `getLogger()`. This method creates a `Logger` object and assigns a `Handler` object to write log messages of the application to the `recipe6.log` file, using the `MyFormatter` formatter. You create the `Logger` object with the static method `getLogger()` of the `Logger` class. This method returns a different object per name that is passed as a parameter. You only created one `Handler` object, so all the `Logger` objects will write their log messages in the same file. You also configured the logger to write all the log messages, regardless of their level.

Finally, you implemented a `Task` object and a main program that writes different log messages in the log file. You used the following methods:

- `entering()`: To write a message with the `FINER` level indicating that a method has started its execution
- `exiting()`: To write a message with the `FINER` level indicating that a method has ended its execution
- `log()`: To write a message with the specified level

There's more...

When you work with a log system, you have to take into consideration two important points:

- **Write the necessary information**: If you write too little information, the logger won't be useful because it won't fulfil its purpose. If you write a lot of information, you will generate large unmanageable log files; this will make it difficult to get the necessary information.
- **Use the adequate level for the messages**: If you write high level information messages or low level error messages, you will confuse the user who will look at the log files. This will make it more difficult to know what happened in an error situation; alternatively, you will have too much of information making it difficult to know the main cause of the error.

There are other libraries that provide a log system that is more complete than the `java.util.logging` package, such as the `Log4j` or `slf4j` libraries. But the `java.util.logging` package is part of the Java API, and all its methods are multithread safe; therefore, we can use it in concurrent applications without problems.

See also

- The *Using non-blocking thread-safe deques*, *Using blocking thread-safe deques*, *Using blocking thread-safe queues ordered by priority*, *Using thread-safe lists with delayed elements* and *Using thread-safe navigable maps* recipes in `Chapter 7`, *Concurrent Collections*

Analyzing concurrent code with FindBugs

Static code analysis tools are a set of tools that analyze the source code of an application while looking for potential errors. These tools, such as Checkstyle, PMD, or FindBugs, have a set of predefined rules of good practices and parse the source code looking for violations of these rules. The objective is to find errors or places that cause poor performance at an early stage, before they are executed in production. Programming languages usually offer such tools, and Java is not an exception. One of the tools that helps analyze Java code is FindBugs. It's an open source tool that includes a series of rules to analyze Java-concurrent code.

In this recipe, you will learn how to use this tool to analyze your Java-concurrent application.

Getting ready

Before you start this recipe, download FindBugs from the project's web page (`http://findbugs.sourceforge.net/`). You can download a standalone application or an Eclipse plugin. In this recipe, I used the standalone version.

At the time of this writing, the actual version of FindBugs (3.0.1) doesn't include support for Java 9. You can download a preview of the 3.1.0 version with support for Java 9 from `https://github.com/findbugsproject/findbugs/releases/tag/3.1.0_preview1`.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task` that extends the `Runnable` interface:

```
public class Task implements Runnable {
```

2. Declare a private `ReentrantLock` attribute named `lock`:

```
private ReentrantLock lock;
```

3. Implement a constructor of the class:

```
public Task(ReentrantLock lock) {
  this.lock=lock;
}
```

4. Implement the `run()` method. Get control of the lock, put the thread to sleep for 2 seconds, and free the lock:

```
@Override
public void run() {
  lock.lock();
  try {
    TimeUnit.SECONDS.sleep(1);
    lock.unlock();
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
}
```

5. Create the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {
  public static void main(String[] args) {
```

6. Declare and create a `ReentrantLock` object named `lock`:

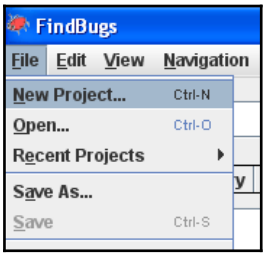
```
ReentrantLock lock=new ReentrantLock();
```

7. Create 10 `Task` objects and 10 threads to execute the tasks. Start the threads by calling the `run()` method:

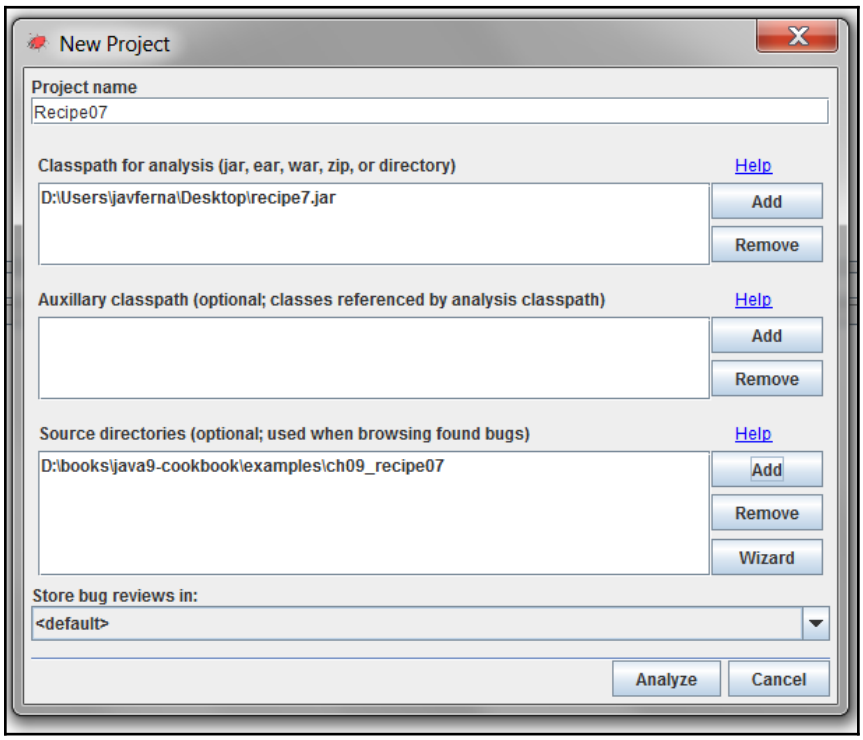
```
  for (int i=0; i<10; i++) {
    Task task=new Task(lock);
    Thread thread=new Thread(task);
    thread.run();
  }
}
```

8. Export the project as a `.jar` file. Call it `recipe7.jar`. Use the menu option of your IDE or the `javac` and `.jar` commands to compile and compress your application.
9. Start the FindBugs standalone application by running the `findbugs.bat` command in Windows or the `findbugs.sh` command in Linux.

10. Create a new project by clicking on the **New Project** option under the **File** menu in the menu bar:



11. The *FindBugs* application shows a window to configure the project. In the **Project name** field, type `Recipe07`. In the **Classpath for analysis field (jar, ear, war, zip, or directory)**, add the `.jar` file with the project. In the **Source directories field (optional; classes used when browsing found bugs)**, add the directory with the source code of the example. Refer to the following screenshot:



12. Click on the **Analyze** button to create the new project and analyze its code.
13. The *FindBugs* application shows the result of the analysis of the code. In this case, it has found two bugs.
14. Click on one of the bugs and you'll see the source code of the bug on the right-hand side panel and the description of the bug in the panel at the bottom of the screen.

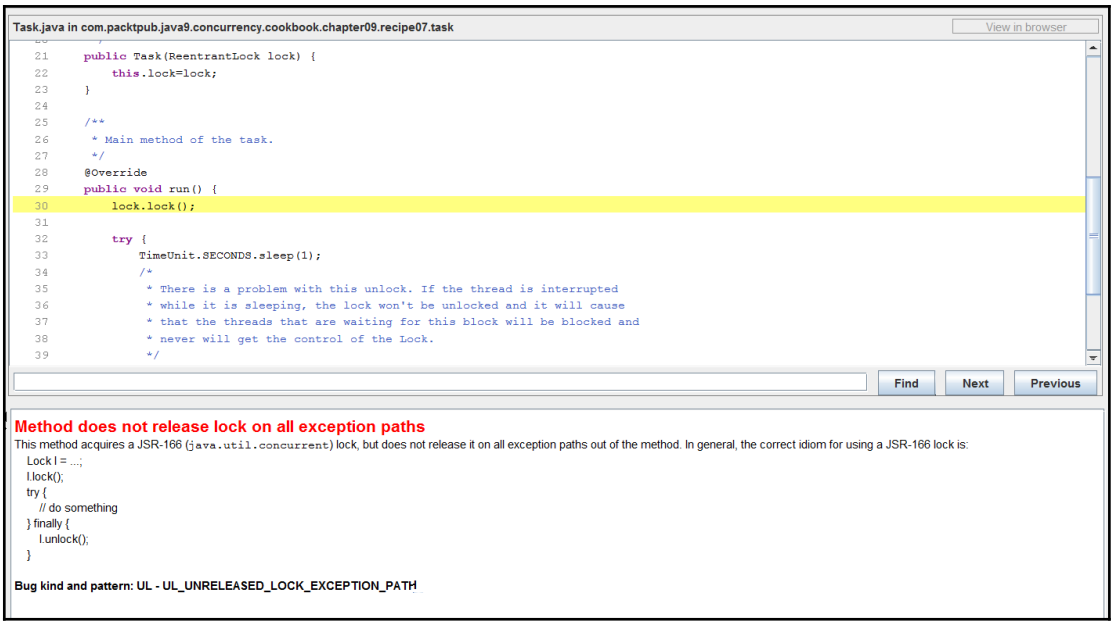
How it works...

The following screenshot shows the result of the analysis by FindBugs:

The analysis has detected the following two potential bugs in the application:

- One of the bugs is detected in the `run()` method of the `Task` class. If an `InterruptedExeption` exception is thrown, the task doesn't free the lock because it won't execute the `unlock()` method. This will probably cause a deadlock situation in the application.
- The other bug is detected in the `main()` method of the `Main` class because you called the `run()` method of a thread directly, not the `start()` method to begin the execution of the thread.

If you double-click on one of the two bugs, you will see detailed information about it. As you have included the source code reference in the configuration of the project, you will also see the source code where the bug was detected. The following screenshot shows you an example of this:



There's more...

Be aware that FindBugs can only detect some problematic situations (related or not to concurrency code). For example, if you delete the `unlock()` call in the `run()` method of the `Task` class and repeat the analysis, FindBugs won't alert you that you will get the lock in the task but you will never be able to free it.

Use the tools of the static code analysis as a form of assistance to improve the quality of your code, but do not expect it to detect all the bugs.

See also

- The *Configuring NetBeans for debugging concurrency code* recipe in this chapter

Configuring Eclipse for debugging concurrency code

Nowadays, almost every programmer, regardless of the programming language in use, create their applications with an IDE. They provide lots of interesting functionalities integrated in the same application, such as:

- Project management
- Automatic code generation
- Automatic documentation generation
- Integration with control version systems
- A debugger to test applications
- Different wizards to create projects and elements of the applications

One of the most helpful features of an IDE is a debugger. Using it, you can execute your application step by step and analyze the values of all the objects and variables of your program.

If you work with Java, Eclipse is one of the most popular IDEs. It has an integrated debugger that allows you to test your applications. By default, when you debug a concurrent application and the debugger finds a breakpoint, it only stops the thread that has the breakpoint while it allows the rest of the threads to continue with their execution. In this recipe, you will learn how to change this configuration to help you test concurrent applications.

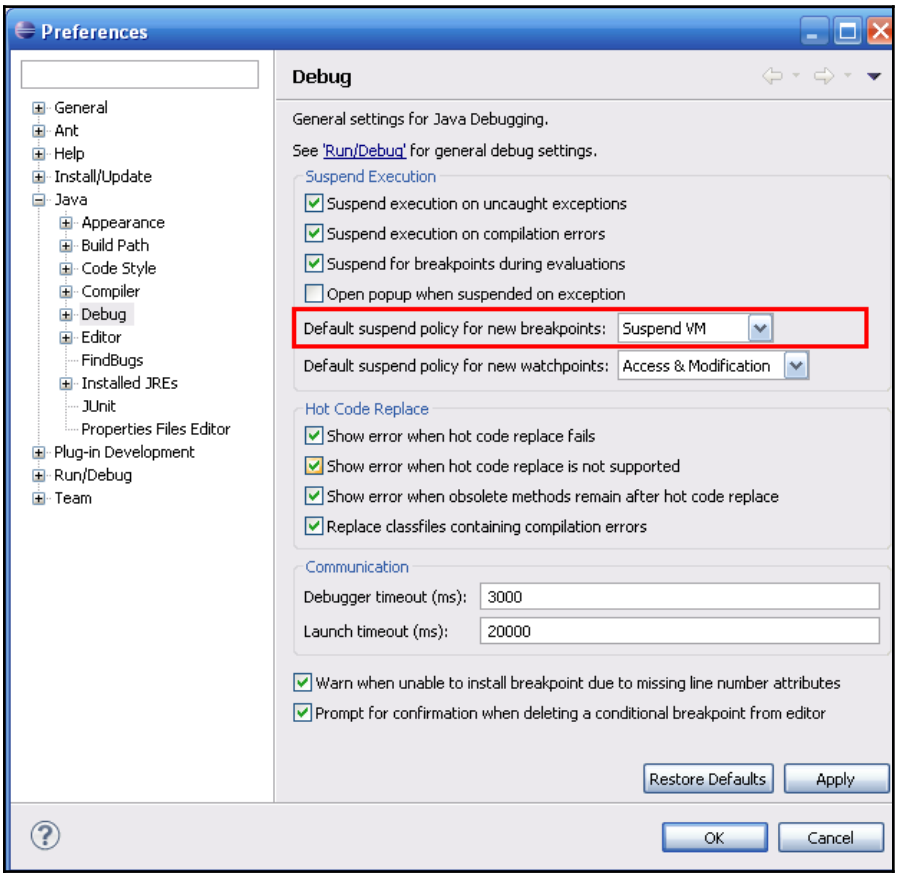
Getting ready

You must have installed the Eclipse IDE. Open it and select a project with a concurrent application implemented, for example, one of the recipes implemented in the book.

How to do it...

Follow these steps to implement the example:

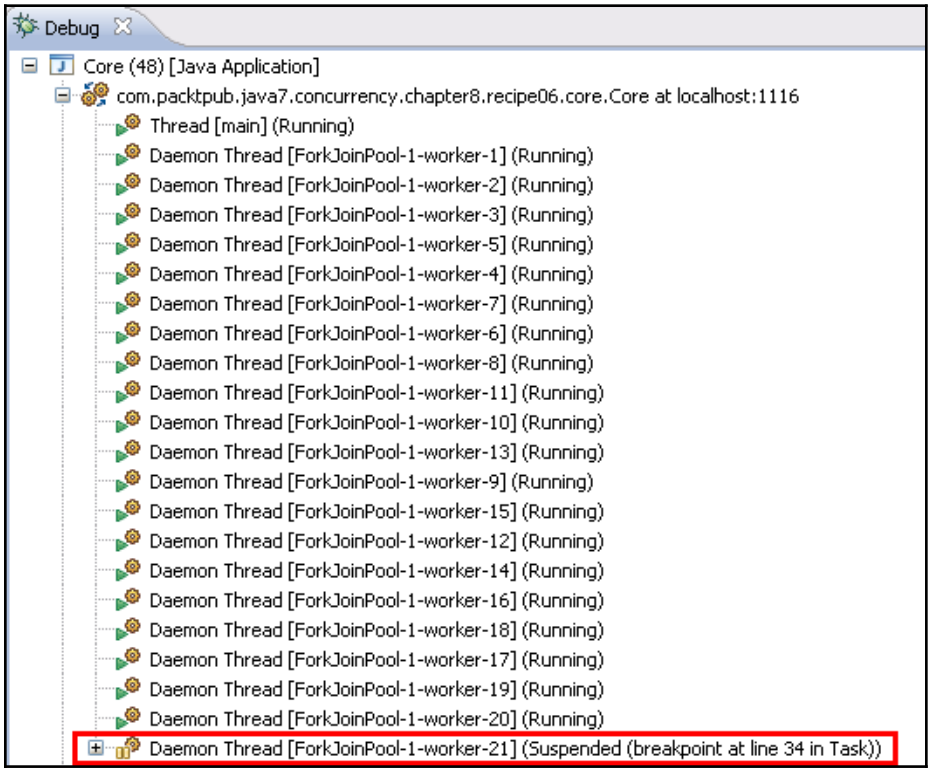
1. Navigate to **Window** | **Preferences**.
2. Expand the **Java** option in the left-hand side menu.
3. Then, select the **Debug** option. The following screenshot illustrates the window:



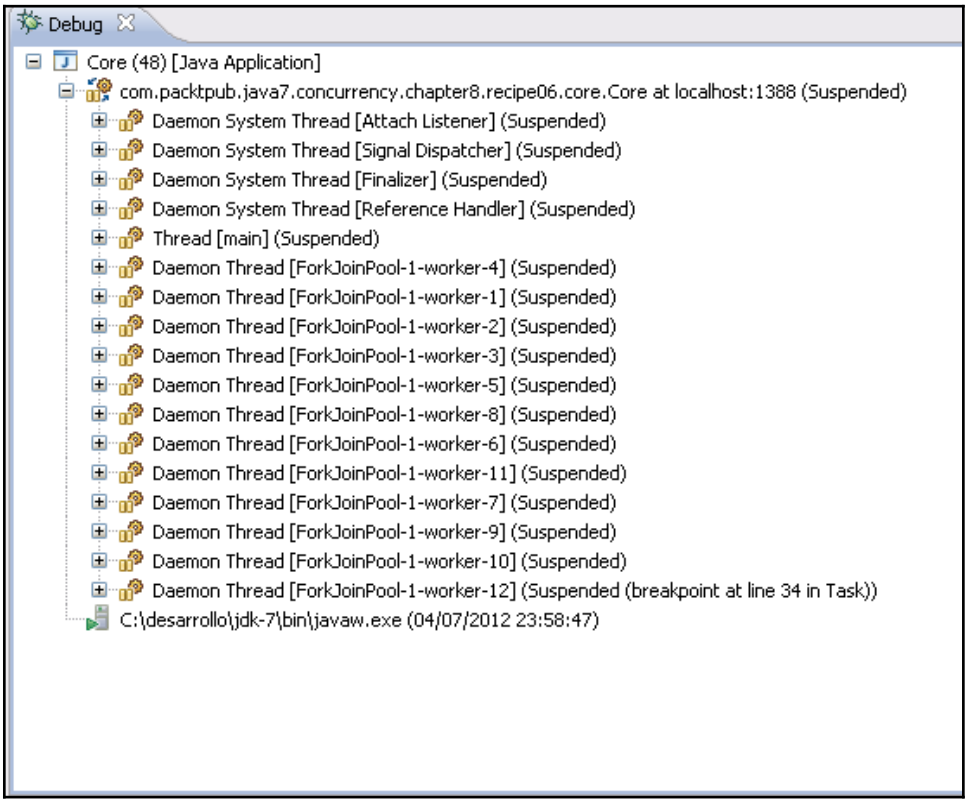
4. Change the value of **Default suspend policy for new breakpoints** from **Suspend Thread** to **Suspend VM** (marked in red in the screenshot).
5. Click on the **OK** button to confirm the change.

How it works...

As mentioned in the introduction of this recipe, by default, when you debug a concurrent Java application in Eclipse and the debug process finds a breakpoint, it only suspends the thread that hits the breakpoint first, but it allows other threads to continue with their execution. The following screenshot shows an example of this:



You can see that only **worker-21** is suspended (marked in red in the screenshot), while the rest of the threads are running. However, while debugging a concurrent application, if you change **Default suspend policy for new breakpoints** to **Suspend VM**, all the threads will suspend their execution and the debug process will hit a breakpoint.. The following screenshot shows an example of this situation:



With the change, you can see that all the threads are suspended. You can continue debugging any thread you want. Choose the suspend policy that best suits your needs.

Configuring NetBeans for debugging concurrency code

Software is necessary to develop applications that work properly, meet the quality standards of the company, and could be easily modified in future (in limited time and cost as low as possible). To achieve this goal, it is essential to use an IDE that can integrate several tools (compilers and debuggers) that facilitate the development of applications under one common interface.

If you work with Java, NetBeans is one of the most popular IDEs. It has an integrated debugger that allows you to test your application.

In this recipe, you will learn how to change the configuration of the Netbeans debugger to help you test concurrent applications.

Getting ready

You should have the NetBeans IDE installed. Open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task1` and specify that it implements the `Runnable` interface:

```
public class Task1 implements Runnable {
```

2. Declare two private `Lock` attributes, named `lock1` and `lock2`:

```
private Lock lock1, lock2;
```

3. Implement the constructor of the class to initialize its attributes:

```
public Task1 (Lock lock1, Lock lock2) {
  this.lock1=lock1;
  this.lock2=lock2;
}
```

4. Implement the `run()` method. First, get control of the `lock1` object using the `lock()` method and write aa message in the console indicating that you have got it:

```
@Override
public void run() {
  lock1.lock();
  System.out.printf("Task 1: Lock 1 locked\n");
```

5. Then, get control of `lock2` using the `lock()` method and write a message in the console indicating that you have got it:

```
lock2.lock();
System.out.printf("Task 1: Lock 2 locked\n");
```

6. Finally, release the two lock objects-first the `lock2` object and then the `lock1` object:

```
  lock2.unlock();
  lock1.unlock();
}
```

7. Create a class named `Task2` and specify that it implements the `Runnable` interface:

```
public class Task2 implements Runnable{
```

8. Declare two private `Lock` attributes, named `lock1` and `lock2`:

```
private Lock lock1, lock2;
```

9. Implement the constructor of the class to initialize its attributes:

```
public Task2(Lock lock1, Lock lock2) {
  this.lock1=lock1;
  this.lock2=lock2;
}
```

10. Implement the `run()` method. First, get control of the `lock2` object using the `lock()` method and write a message in the console indicating that you have got it:

```
@Override
public void run() {
  lock2.lock();
  System.out.printf("Task 2: Lock 2 locked\n");
```

11. Then, get control of `lock1` using the `lock()` method and write a message in the console indicating that you have got it:

```
lock1.lock();
System.out.printf("Task 2: Lock 1 locked\n");
```

12. Finally, release the two lock objects-first `lock1` and then `lock2`:

```
  lock1.unlock();
  lock2.unlock();
}
```

13. Implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
```

14. Create two lock objects named `lock1` and `lock2`:

```
Lock lock1, lock2;
lock1=new ReentrantLock();
lock2=new ReentrantLock();
```

15. Create a `Task1` object named `task1`:

```
Task1 task1=new Task1(lock1, lock2);
```

16. Create a `Task2` object named `task2`:

```
Task2 task2=new Task2(lock1, lock2);
```

17. Execute both the tasks using two threads:

```
Thread thread1=new Thread(task1);
Thread thread2=new Thread(task2);

thread1.start();
thread2.start();
```

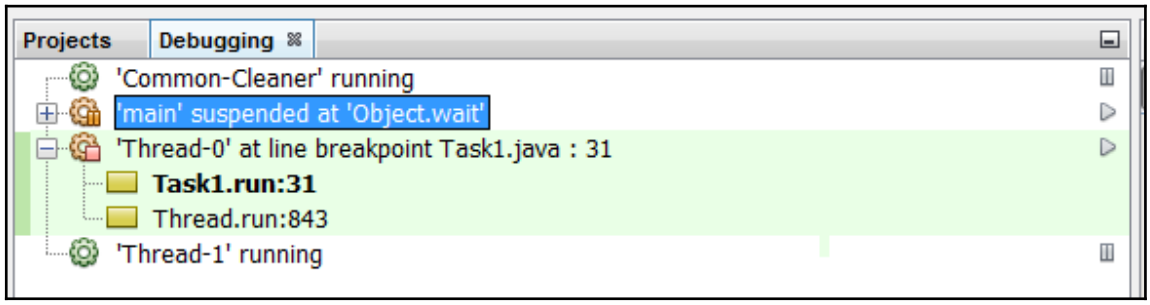
18. When the two tasks finish their execution, write a message in the console every 500 milliseconds. Use the `isAlive()` method to check whether a thread has finished its execution:

```
while ((thread1.isAlive()) &&(thread2.isAlive())) {
  System.out.println("Main: The example is"+ "running");
  try {
```

```
            TimeUnit.MILLISECONDS.sleep(500);
          } catch (InterruptedException ex) {
            ex.printStackTrace();
          }
        }
```

19. Add a breakpoint in the first call to the `printf()` method of the `run()` method of the `Task1` class.
20. Debug the program. You will see the **Debugging** window in the top left-hand side corner of the main NetBeans window. The next screenshot illustrates the window with the thread that executes the `Task1` object. The thread is waiting in the breakpoint. The other threads of the application are running:

21. Pause the execution of the main thread. Select the thread, right-click on it, and select the **Suspend** option. The following screenshot shows the new appearance of the **Debugging** window. Refer to the following screenshot:



22. Resume the two paused threads. Select each thread, right-click on them, and select the **Resume** option.

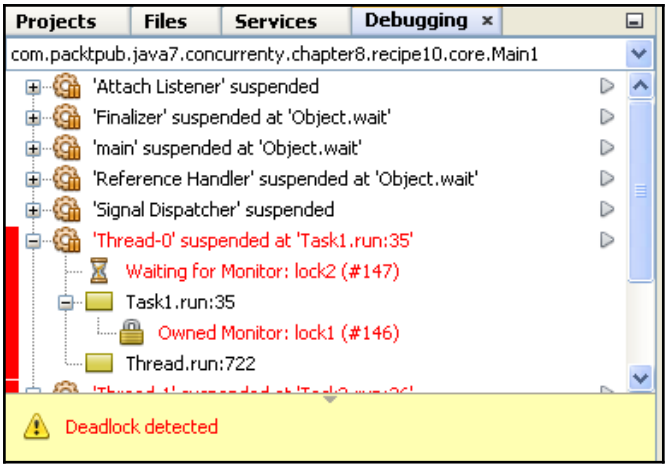
How it works...

While debugging a concurrent application using NetBeans, when the debugger hits a breakpoint, it suspends the thread that hit the breakpoint and shows the **Debugging** window in the top left-hand side corner with the threads that are currently running. You can use the window to pause or resume the threads that are currently running, using the **Pause** or **Resume** options. You can also see the values of the variables or attributes of the threads using the **Variables** tab.

NetBeans also includes a deadlock detector. When you select the **Check for Deadlock** option in the **Debug** menu, NetBeans performs an analysis of the application that you're debugging to determine whether there's a deadlock situation. This example presents a clear deadlock. The first thread gets `lock1` first and then `lock2`. The second thread gets the locks in reverse manner. The breakpoint inserted provokes the deadlock, but if you use the NetBeans deadlock detector, you'll not find anything. Therefore, this option should be used with caution. Change the locks used in both the tasks by the `synchronized` keyword and debug the program again. The code of `Task1` is as follows:

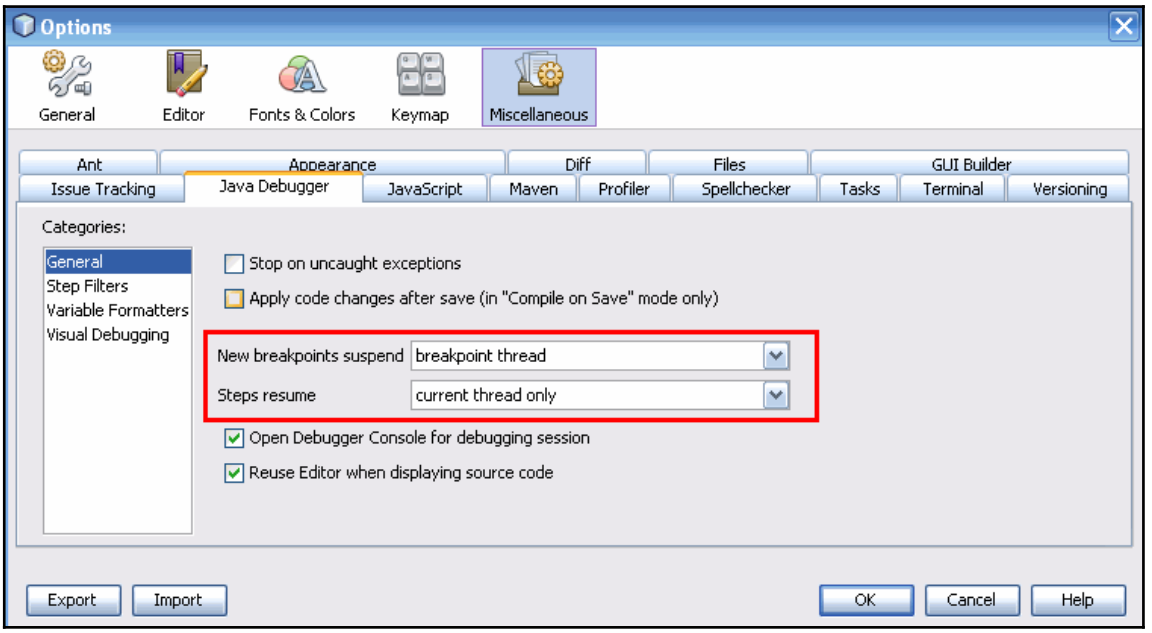
```
@Override
public void run() {
  synchronized(lock1) {
    System.out.printf("Task 1: Lock 1 locked\n");
    synchronized(lock2) {
      System.out.printf("Task 1: Lock 2 locked\n");
    }
  }
}
```

The code of the `Task2` class will be analogous to this, but it changes the order of the locks. If you debug the example again, you will obtain a deadlock one more time. However, in this case, it's detected by the deadlock detector, as you can see in the following screenshot:



There's more...

There are options to control the debugger. Select **Options** from the **Tools** menu. Then, select the **Miscellaneous** option and the **Java Debugger** tab. The following screenshot illustrates this window:



There are two options in the window that control the behavior described earlier:

- **New breakpoints suspend**: With this option, you can configure the behavior of NetBeans, which finds a breakpoint in a thread. You can suspend only that thread that has a breakpoint or all the threads of the application.
- **Steps resume**: With this option, you can configure the behavior of NetBeans when you resume a thread. You can resume only the current thread or all the threads.

Both the options have been marked in the screenshot presented earlier.

See also

- The *Configuring Eclipse for debugging concurrency code* recipe in this chapter

Testing concurrency code with MultithreadedTC

`MultithreadedTC` is a Java library for testing concurrent applications. Its main objective is to solve the problem of concurrent applications being non-deterministic. You can't control the order of execution of the different threads that form the application. For this purpose, it includes an internal **metronome**. These testing threads are implemented as methods of a class.

In this recipe, you will learn how to use the `MultithreadedTC` library to implement a test for `LinkedTransferQueue`.

Getting ready

Download the MultithreadedTC library from `https://code.google.com/archive/p/multithreadedtc/` and the JUnit library, version 4.10, from `http://junit.org/junit4/`. Add the `junit-4.10.jar` and `MultithreadedTC-1.01.jar` files to the libraries of the project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `ProducerConsumerTest` that extends the `MultithreadedTestCase` class:

```
public class ProducerConsumerTest extends MultithreadedTestCase {
```

2. Declare a private `LinkedTransferQueue` attribute parameterized by the `String` class named `queue`:

```
private LinkedTransferQueue<String> queue;
```

3. Implement the `initialize()` method. This method won't receive any parameters and will return no value. It will call the `initialize()` method of its parent class and then initialize the queue attribute:

```
@Override
public void initialize() {
  super.initialize();
  queue=new LinkedTransferQueue<String>();
  System.out.printf("Test: The test has been initialized\n");
}
```

4. Implement the `thread1()` method. It will implement the logic of the first consumer. Call the `take()` method of the queue and then write the returned value in the console:

```
public void thread1() throws InterruptedException {
  String ret=queue.take();
  System.out.printf("Thread 1: %s\n",ret);
}
```

5. Implement the `thread2()` method. It will implement the logic of the second consumer. First wait until the first thread has slept in the `take()` method. To put the thread to sleep, use the `waitForTick()` method. Then, call the `take()` method of the queue and write the returned value in the console:

```
public void thread2() throws InterruptedException {
  waitForTick(1);
  String ret=queue.take();
  System.out.printf("Thread 2: %s\n",ret);
}
```

6. Implement the `thread3()` method. It will implement the logic of a producer. First wait until the two consumers are blocked in the `take()` method; block this method using the `waitForTick()` method twice. Then, call the `put()` method of the queue to insert two strings in the queue:

```
 public void thread3() {
  waitForTick(1);
  waitForTick(2);
  queue.put("Event 1");
  queue.put("Event 2");
  System.out.printf("Thread 3: Inserted two elements\n");
}
```

7. Finally, implement the `finish()` method. Write a message in the console to indicate that the test has finished its execution. Check that the two events have been consumed (so the size of the queue is `0`) using the `assertEquals()` method:

```
public void finish() {
  super.finish();
  System.out.printf("Test: End\n");
  assertEquals(true, queue.size()==0);
  System.out.printf("Test: Result: The queue is empty\n");
}
```

8. Next, implement the main class of the example by creating a class named `Main` with a `main()` method:

```
public class Main {
  public static void main(String[] args) throws Throwable {
```

9. Create a `ProducerConsumerTest` object named `test`:

```
ProducerConsumerTest test=new ProducerConsumerTest();
```

10. Execute the test using the `runOnce()` method of the `TestFramework` class:

```
System.out.printf("Main: Starting the test\n");
TestFramework.runOnce(test);
System.out.printf("Main: The test has finished\n");
```

How it works...

In this recipe, you implemented a test for the `LinkedTransferQueue` class using the `MultithreadedTC` library. You can implement a test in any concurrent application or class using this library and its metronome. In the example, you implemented the classical producer/consumer problem with two consumers and a producer. You wanted to test that the first `String` object introduced in the buffer is consumed by the first consumer that arrives at the buffer, and the second `String` object introduced in the buffer is consumed by the second consumer that arrives at the buffer.

The `MultithreadedTC` library is based on the JUnit library, which is the most often used library to implement unit tests in Java. To implement a basic test using the `MultithreadedTC` library, you have to extend the `MultithreadedTestCase` class. This class extends the `junit.framework.AssertJUnit` class that includes all the methods to check the results of the test. It doesn't extend the `junit.framework.TestCase` class, so you can't integrate MultithreadedTC tests with other JUnit tests.

Then, you can implement the following methods:

- `initialize()`: The implementation of this method is optional. It's executed when you start the test, so you can use it to initialize objects that are using the test.
- `finish()`: The implementation of this method is optional. It's executed when the test has finished. You can use it to close or release resources used during the test or to check the results of the test.
- Methods that implement the test: These methods have the main logic of the test you implement. They have to start with the `thread` keyword, followed by a string, for example, `thread1()`.

To control the order of execution of threads, you used the `waitForTick()` method. This method receives an integer value as a parameter and puts the thread that is executing the method to sleep until all the threads that are running in the test are blocked. When they are blocked, the `MultithreadedTC` library resumes the threads that are blocked by a call to the `waitForTick()` method.

The integer you pass as a parameter of the `waitForTick()` method is used to control the order of execution. The metronome of the `MultithreadedTC` library has an internal counter. When all the threads are blocked, the library increments this counter to the next number specified in the `waitForTick()` calls that are blocked.

Internally, when the `MultithreadedTC` library has to execute a test, first it executes the `initialize()` method. Then it creates a thread per method that starts with the `thread` keyword (in your example, the methods `thread1()`, `thread2()`, and `thread3()`). When all the threads have finished their execution, it executes the `finish()` method. To execute the test, you used the `runOnce()` method of the `TestFramework` class.

There's more...

If the `MultithreadedTC` library detects that all the threads of the test are blocked except in the `waitForTick()` method, the test is declared to be in a deadlock state and a `java.lang.IllegalStateException` exception is thrown.

See also

- The *Analyzing concurrent code with FindBugs* recipe in this chapter

Monitoring with JConsole

JConsole is a monitoring tool that follows the JMX specification that allows you to get information about the execution of an application as the number of threads, memory use, or class loading. It is included with the JDK and it can be used to monitor local or remote applications. In this recipe, you will learn how to use this tool to monitor a simple concurrent application.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `Task` and specify the `Runnable` interface. Implement the `run()` method to write the message in the console during 100 seconds:

```
public class Task implements Runnable {

  @Override
  public void run() {

    Date start, end;
    start = new Date();
    do {
      System.out.printf("%s: tick\n",
                        Thread.currentThread().getName());
      end = new Date();
    } while (end.getTime() - start.getTime() < 100000);
  }
}
```

2. Implement the `Main` class with the `main()` method. Create 10 `Task` objects to create 10 threads. Start them and wait for their finalization using the `join()` method:

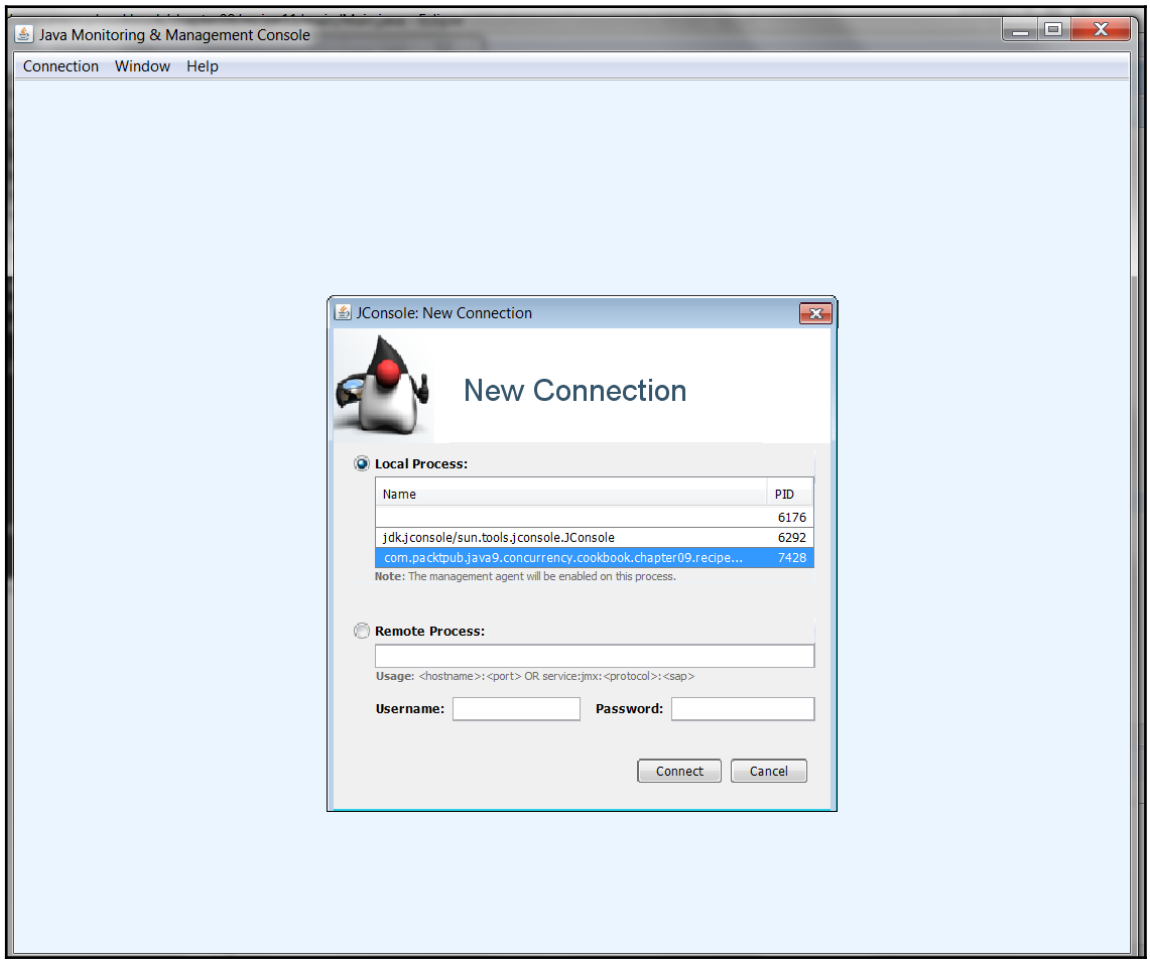
```
public class Main {
  public static void main(String[] args) {

    Thread[] threads = new Thread[10];

    for (int i=0; i<10; i++) {
      Task task=new Task();
      threads[i]=new Thread(task);
      threads[i].start();
    }

    for (int i=0; i<10; i++) {
      try {
        threads[i].join();
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
  }
}
```

3. Open a console window and execute the `JConsole` application. It's included in the bin directory of the JDK-9 installation:

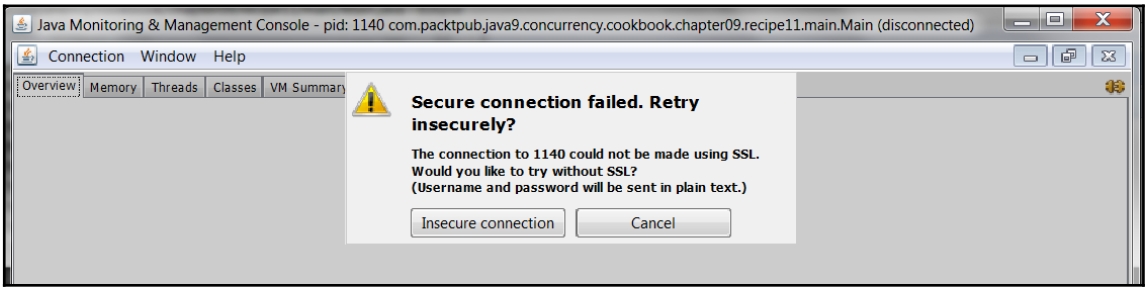
How it works...

In this recipe, we implemented a very simple example: running 10 threads for 100 seconds. These are threads that write messages in the console.

When you execute JConsole, you will see a window that shows all the Java applications that are running in your system. You can choose the one you want to monitor. The window will be similar to the following one:



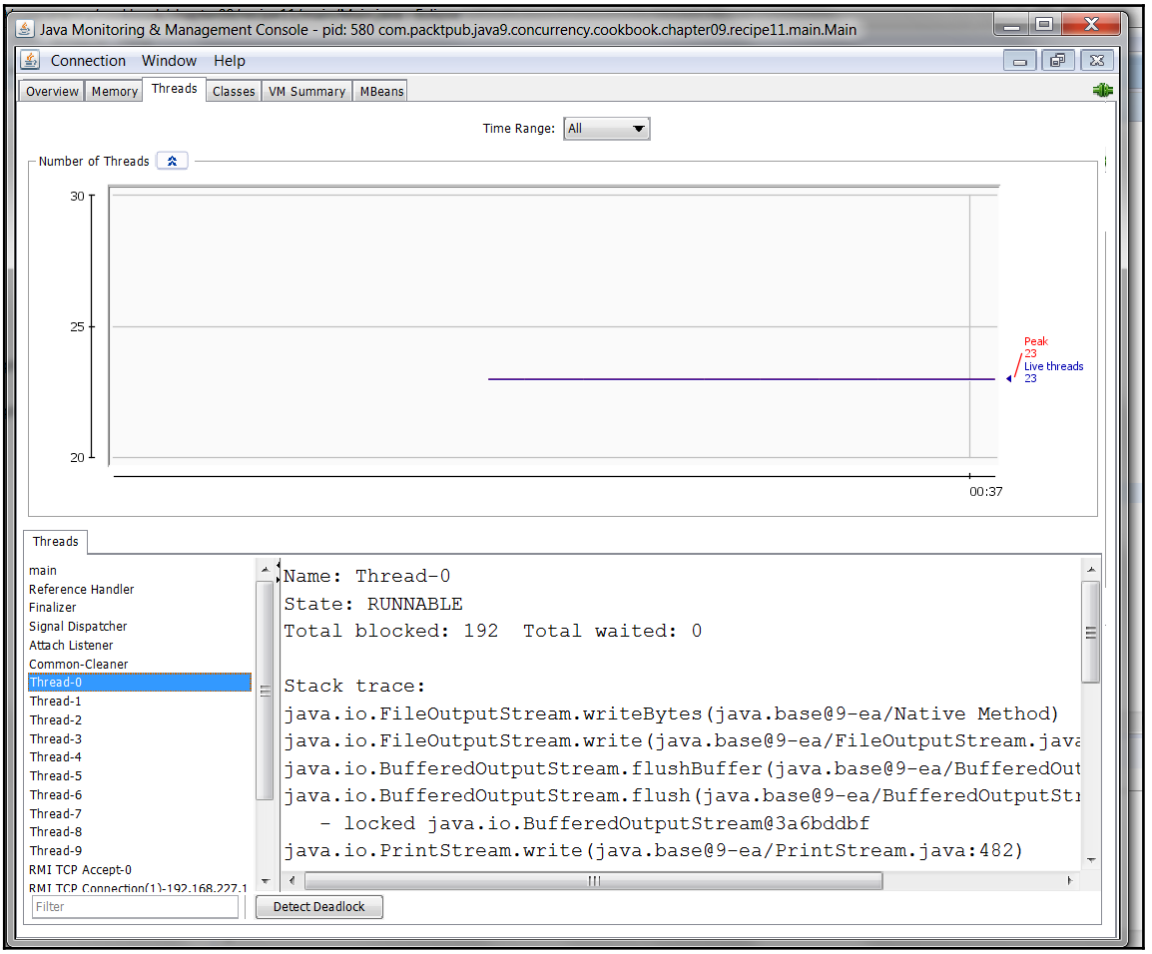
In this case, we select our sample app and click on the **Connect** button. Then, you will be asked to establish an insecure connection with the application, with a dialog similar to the following one:



Click on the **Insecure connection** button. JConsole will show you information about your application using six tabs:

- The **Overview** tab provides an overview of memory use, the number of threads running in the application, the number of objects created, and CPU usage of the application.
- The **Memory** tab shows the amount of memory used by the application. It has a combo where you can select the type of memory you want to monitor (heap, non-heap, or pools).
- The **Threads** tab shows you information about the number of threads in the application and detailed information about each thread.
- The **Classes** tab shows you information about the number of objects loaded in the application.
- The **VW Summary** tab provides a summary of the JVM running the application.
- The **MBeans** tab shows you information about the managed beans of the application.

The threads tab is similar to the following one:



It has two different parts. In the upper part, you have real-time information about the **Peak** number of threads (with a red line) and the number of **Live Threads** (with a blue line). In the lower part, we have a list of active threads. When you select one of these threads, you will see detailed information about that thread, including its status and the actual stack trace.

There's more...

You can use other applications to monitor applications that run Java. For example, you can use VisualVM included with the JDK. You can obtain necessary information about visualvm at `https://visualvm.github.io`.

See also

- The *Testing concurrency code with MultithreadedTC* recipe in this chapter